

Python Data Analysis Workbook

NumPy • pandas • DataFrames • Matplotlib

50 Hands-On Exercises Using a Real Dataset:

"Education in India" — District-Wise School Data, 2015-16

20 Low Complexity • 15 Medium Complexity • 15 High Complexity

For use in Jupyter Notebook

How to Use This Workbook

1. Get the Dataset

2.

Download the file '2015_16_Districtwise.csv' from the Kaggle dataset 'Education in India' (rajanand/education-in-india).

https://www.kaggle.com/datasets/rajanand/education-in-india?select=2015_16_Districtwise.csv

Place it in the same folder as your Jupyter notebook, or note its full file path.

2. Set Up Your Notebook

Create a new Jupyter Notebook (Anaconda / Google Colab / VS Code all work).

In the first cell run: `pip install pandas numpy matplotlib` (skip if already installed).

Import the three libraries at the top of your notebook: `import numpy as np / import pandas as pd / import matplotlib.pyplot as plt`

Add `%matplotlib inline` as the very first line if using classic Jupyter, so charts render below each cell.

3. About the Column Names

Column names referenced below (e.g. STATNAME, DISTNAME, OVERALL_LI, TOTPOPULAT, SEXRATIO, SCHTOT, TCHTOT, P_URB_POP, GROWTHRATE) reflect the common DISE 2015-16 district-wise schema. Column names can vary slightly by dataset version, so Exercise 1 asks you to print `df.columns` and confirm exact names before proceeding.

Complete Exercise 1 first — it has you print the exact column list, which you'll refer back to throughout this workbook.

4. How the Exercises Are Organized

Low Complexity (1-20): loading data, exploring structure, basic filtering/sorting/grouping, first NumPy arrays, and simple single-panel charts.

Medium Complexity (21-35): multi-column aggregation, merging, pivot tables, `apply()/lambda` logic, correlation, and multi-panel charts.

High Complexity (36-50): composite indices, outlier detection, weighted statistics, bootstrap resampling, residual analysis, and full dashboards.

Each exercise lists the pandas/NumPy/matplotlib concepts it practices and leaves space for your own notes and insights.

Try to write your own code before checking documentation — struggling productively is part of learning!

PART 1 — LOW COMPLEXITY (Exercises 1–20)

Goal: get comfortable loading data, inspecting it, and producing your first charts. These exercises map directly to core pandas/NumPy/matplotlib basics.

Exercise 1 [Low] — Load the Dataset and First Look

Concepts practiced: `pandas.read_csv` • `head()` • `tail()` • `shape`

1. Import numpy as np, pandas as pd, and matplotlib.pyplot as plt.
2. Load 2015_16_Districtwise.csv into a DataFrame called df using `pd.read_csv()`.
3. Print `df.shape` and describe in a comment what the two numbers mean.
4. Display the first 10 rows using `df.head(10)` and the last 5 rows using `df.tail(5)`.
5. Print `df.columns.tolist()` so you have the exact column names for the rest of this workbook.

Your notes / observations:

Exercise 2 [Low] — Explore Structure with `info()` and `describe()`

Concepts practiced: `df.info()` • `df.describe()` • `dtypes`

6. Run `df.info()` and note how many columns are numeric (int64/float64) vs object (text).
7. Run `df.describe()` and identify the column with the largest maximum value.
8. Use `df.dtypes` to list the data type of every column.
9. Write one sentence describing what the 'count' row in `describe()` tells you about missing data.

Your notes / observations:

Exercise 3 [Low] — Column Inventory: Numeric vs Categorical

Concepts practiced: `select_dtypes` • `list comprehension`

10. Use `df.select_dtypes(include='number')` to get only numeric columns.
11. Use `df.select_dtypes(include='object')` to get only text/categorical columns.
12. Print how many columns fall into each group.
13. Identify which categorical column uniquely identifies a district (hint: DISTNAME + STATNAME).

Your notes / observations:

Exercise 4 [Low] — Missing Value Audit

Concepts practiced: `isnull()` • `sum()` • `sort_values()`

14. Use `df.isnull().sum()` to count missing values per column.
15. Sort the missing-value counts in descending order.
16. Print the total number of missing cells in the whole DataFrame.

17. Print the percentage of missing values for the 3 columns with the most gaps.

Your notes / observations:

Exercise 5 [Low] — Selecting Specific Columns

Concepts practiced: column selection • double brackets

18. Create a smaller DataFrame called `lit_df` containing only `STATNAME`, `DISTNAME` and the overall literacy column.
19. Display the first 15 rows of `lit_df`.
20. Rename the literacy column to `LITERACY_RATE` using `df.rename()`.
21. Confirm the rename worked by printing `lit_df.columns`.

Your notes / observations:

Exercise 6 [Low] — Filtering Rows with Conditions

Concepts practiced: boolean masking • comparison operators

22. Filter `df` to keep only districts where the overall literacy rate is greater than 90.
23. Print how many districts satisfy this condition.
24. Filter `df` again to find districts where literacy is below 60 - how many are there?
25. Combine both conditions using `&` to confirm no district can satisfy both at once.

Your notes / observations:

Exercise 7 [Low] — Sorting the Data

Concepts practiced: `sort_values()` • ascending/descending

26. Sort the DataFrame by overall literacy rate in descending order.
27. Print the top 10 districts by literacy.
28. Sort by total population in ascending order and print the 5 least populated districts.
29. Sort by two columns at once: `STATNAME` (A-Z) then literacy rate (high to low).

Your notes / observations:

Exercise 8 [Low] — Grouping by State

Concepts practiced: `groupby()` • `mean()`

30. Group the DataFrame by `STATNAME` and compute the mean overall literacy rate per state.
31. Sort the resulting Series to find the 5 states with the highest average literacy.

32. Find the 5 states with the lowest average literacy.
33. Print the difference between the highest and lowest state averages.

Your notes / observations:

Exercise 9 [Low] — Counting Districts per State

Concepts practiced: `value_counts()`

34. Use `df['STATNAME'].value_counts()` to count the number of districts per state.
35. Print which state has the most districts and how many.
36. Print which state has the fewest districts and how many.
37. Convert the `value_counts()` result into a DataFrame with columns `STATE` and `DISTRICT_COUNT`.

Your notes / observations:

Exercise 10 [Low] — First Steps with NumPy Arrays

Concepts practiced: `np.array` • `min/max/mean/median/std`

38. Convert the total population column into a NumPy array called `pop_arr`.
39. Compute `np.min`, `np.max`, `np.mean`, `np.median` and `np.std` of `pop_arr`.
40. Print all five statistics with clear labels.
41. Compute the range (`max - min`) using plain NumPy arithmetic, not a separate function.

Your notes / observations:

Exercise 11 [Low] — Creating a Derived Column

Concepts practiced: *arithmetic on columns* • *vectorized operations*

42. Create a new column `URBAN_POP_EST = TOTPOPULAT * (P_URB_POP / 100)`.
43. Round the new column to the nearest whole number using `np.round()`.
44. Display `STATNAME`, `DISTNAME` and `URBAN_POP_EST` for the top 5 districts by this new column.
45. Explain in a comment why this is only an estimate, not an exact figure.

Your notes / observations:

Exercise 12 [Low] — Practicing `loc` and `iloc`

Concepts practiced: `.loc[]` • `.iloc[]`

46. Use `df.iloc[10:20, 2:6]` to select rows 10-19 and columns 2-5.

47. Use `df.loc[df['STATNAME'] == 'KERALA']` to select all Kerala districts (adjust the exact state string as needed).
48. Use `.loc` to select only the `STATNAME` and literacy columns for rows where population exceeds 2,000,000.
49. Explain in a comment the difference between `.loc` (label-based) and `.iloc` (position-based).

Your notes / observations:

Exercise 13 [Low] — Dropping Unneeded Columns

Concepts practiced: `drop()` • `axis=1`

50. Create a copy of `df` called `df_clean`.
51. Drop the state and district numeric code columns (e.g. `STATCD`, `DISTCD`) from `df_clean`.
52. Print `df_clean.shape` before and after dropping to confirm the column count decreased.
53. Save `df_clean` back into a new CSV file called `districtwise_clean.csv` using `to_csv()`.

Your notes / observations:

Exercise 14 [Low] — Finding State-Level Extremes

Concepts practiced: `groupby()` • `idxmax()/idxmin()`

54. Group by `STATNAME` and compute the mean sex ratio (females per 1000 males) per state.
55. Use `idxmax()` to find the state with the highest average sex ratio.
56. Use `idxmin()` to find the state with the lowest average sex ratio.
57. Print both results in a readable sentence, e.g. 'X has the highest average sex ratio at Y'.

Your notes / observations:

Exercise 15 [Low] — Bar Chart: Top 10 States by Literacy

Concepts practiced: `matplotlib bar chart` • `groupby` + `plot`

58. Compute average literacy per state and select the top 10 states.
59. Create a horizontal bar chart (`plt.barh`) showing these 10 states, sorted from highest to lowest.
60. Add a title, x-axis label ('Average Literacy Rate %') and y-axis label ('State').
61. Use `plt.tight_layout()` so state names are not cut off, then `plt.show()`.

Your notes / observations:

Exercise 16 [Low] — Histogram of Literacy Distribution

Concepts practiced: `plt.hist()` • `bins` parameter

62. Plot a histogram of the overall literacy column across all ~680 districts using 20 bins.
63. Add a title 'Distribution of District Literacy Rates' and label both axes.
64. Add a vertical line (`plt.axvline`) at the national average literacy rate.
65. In a comment, describe whether the distribution looks roughly symmetric or skewed.

Your notes / observations:

Exercise 17 [Low] — Pie Chart of District Share by Top States

Concepts practiced: `plt.pie()` • `value_counts()`

66. Find the 5 states with the most districts using `value_counts().head(5)`.
67. Create a pie chart showing each of these 5 states' share of districts, with percentage labels (`autopct='%1.1f%%'`).
68. Add a title 'Top 5 States by Number of Districts'.
69. Add a legend showing the state names.

Your notes / observations:

Exercise 18 [Low] — Gender Literacy Gap Flag

Concepts practiced: `boolean column` • `sum()`

70. Create a boolean column `GENDER_GAP_FLAG` that is True when the male literacy rate is higher than the female literacy rate in a district.
71. Count how many districts have this flag set to True out of the total number of districts.
72. Print this count as a percentage of all districts.
73. Print the 5 districts with the largest positive gap (male minus female).

Your notes / observations:

Exercise 19 [Low] — Using `np.where` for Classification

Concepts practiced: `np.where()` • `conditional column creation`

74. Use `np.where()` to create a column `GENDER_GAP_LABEL` that stores 'Male Higher' when male literacy exceeds female literacy, and 'Female Higher or Equal' otherwise.
75. Print `value_counts()` of this new column.
76. Compare this result to Exercise 18 - do the counts match?
77. Print 3 example districts for each label.

Your notes / observations:

Exercise 20 [Low] — Basic Summary Report

Concepts practiced: print formatting • f-strings

78. Using everything learned so far, print a short 6-line summary report using f-strings covering: total districts, total states, national average literacy, state with highest literacy, state with lowest literacy, and total missing values in the dataset.
79. Format all percentages to 2 decimal places.
80. This exercise has no charts - focus on clean, readable printed output.

Your notes / observations:

PART 2 — MEDIUM COMPLEXITY (Exercises 21–35)

Goal: combine multiple pandas/NumPy operations together, build multi-panel visualizations, and start drawing real insights by comparing groups.

Exercise 21 [Medium] — Quantifying the Gender Literacy Gap

Concepts practiced: derived column • sort_values() • head()

81. Create a column `LIT_GENDER_GAP` = male literacy - female literacy for every district.
82. Sort districts by this gap in descending order and display the top 10 (largest male advantage).
83. Sort ascending and display the top 10 districts where female literacy exceeds male literacy the most.
84. Compute the average `LIT_GENDER_GAP` nationally and per state, and report the 3 states with the smallest average gap.

Your notes / observations:

Exercise 22 [Medium] — Multi-Statistic Aggregation with `agg()`

Concepts practiced: groupby().agg() • multiple functions

85. Group by `STATNAME` and use `.agg(['mean','min','max','std'])` on the overall literacy column in one call.
86. Sort the result by the 'mean' column descending.
87. Identify which state has the largest standard deviation (most internal variation between its districts) and which has the smallest.
88. Print a short interpretation of what a high std means for a state's district-level equality in literacy.

Your notes / observations:

Exercise 23 [Medium] — Building a Pivot Table

Concepts practiced: pd.pivot_table()

89. Create a pivot table with `STATNAME` as the index and mean values of overall literacy, total population, and sex ratio as values.
90. Sort the pivot table by mean literacy descending.
91. Round all values in the pivot table to 1 decimal place.
92. Export the pivot table to a CSV file called `state_summary.csv`.

Your notes / observations:

Exercise 24 [Medium] — Merging in a Custom Region Mapping

Concepts practiced: pd.merge() • dictionary to DataFrame

93. Manually build a small `DataFrame` called `region_map` with two columns, `STATNAME` and `REGION`, assigning each state to North, South, East, West or Northeast (your own reasonable grouping).

94. Merge region_map onto df using pd.merge(df, region_map, on='STATNAME', how='left').
95. Group the merged DataFrame by REGION and compute mean literacy per region.
96. Create a bar chart comparing average literacy across the 5 regions.

Your notes / observations:

Exercise 25 [Medium] — Classifying Districts with apply() and a Lambda

Concepts practiced: apply() • lambda • binning logic

97. Write a lambda (or small function) that returns 'Low' if literacy < 60, 'Medium' if 60-80, and 'High' if > 80.
98. Apply this function to the literacy column using df['col'].apply() to create a new column LIT_BAND.
99. Print value_counts() of LIT_BAND to see how districts are distributed across the 3 bands.
100. Create a bar chart showing the count of districts in each band.

Your notes / observations:

Exercise 26 [Medium] — Handling Missing Data: fillna vs dropna

Concepts practiced: fillna() • dropna() • shape comparison

101. Pick 2 numeric columns that contain missing values.
102. Create df_filled by filling missing values in those columns with the column mean using fillna().
103. Create df_dropped by removing any row with a missing value in those columns using dropna().
104. Print df.shape, df_filled.shape and df_dropped.shape side by side and explain the difference in row counts.

Your notes / observations:

Exercise 27 [Medium] — Z-Scores and Simple Outlier Flags

Concepts practiced: np.mean • np.std • z-score formula

105. Manually compute the z-score for every district's literacy rate using the formula $(x - \text{mean}) / \text{std}$ with NumPy (do not use scipy).
106. Add the z-scores as a new column Z_LITERACY.
107. Flag any district with $|Z_LITERACY| > 3$ as an outlier and print those rows.
108. Comment on whether any outliers were found and whether they seem like data errors or genuinely extreme districts.

Your notes / observations:

Exercise 28 [Medium] — Correlation Matrix of Key Variables

Concepts practiced: df.corr() • correlation interpretation

109. Select 6-8 key numeric columns (literacy, population, sex ratio, schools, teachers, urban %, growth rate).
110. Compute the correlation matrix using `df[cols].corr()`.
111. Identify the 3 strongest positive correlations and the 3 strongest negative correlations (excluding self-correlation of 1.0).
112. Print a one-sentence interpretation for the single strongest correlation you found.

Your notes / observations:

Exercise 29 [Medium] — Scatter Plot: Population vs Number of Schools

Concepts practiced: `plt.scatter()` • color encoding

113. Create a scatter plot with total population on the x-axis and total number of schools on the y-axis, one point per district.
114. Color the points using literacy rate as a colormap (use the `c` and `cmap` parameters) and add a colorbar.
115. Add a title and axis labels.
116. Comment on whether more populated districts generally have proportionally more schools.

Your notes / observations:

Exercise 30 [Medium] — A 2x2 Grid of Distributions

Concepts practiced: `plt.subplots()` • multiple histograms

117. Create a 2x2 grid of subplots using `plt.subplots(2, 2, figsize=(12,8))`.
118. In each of the 4 panels plot a histogram: literacy, sex ratio, total population, and one more numeric column of your choice.
119. Give each subplot its own title.
120. Use `plt.tight_layout()` to keep the layout clean before showing the figure.

Your notes / observations:

Exercise 31 [Medium] — Ranking States by an Infrastructure Composite

Concepts practiced: `groupby().mean()` • combining multiple columns

121. Pick 3-4 school infrastructure columns available in the dataset (e.g. electricity, drinking water, toilets, boundary wall availability - use whichever exist after checking `df.columns`).
122. Compute a simple average infrastructure score per district by averaging these columns (converting to the same 0-100 scale if needed).
123. Group by state and rank states by their mean infrastructure score.
124. Print the top 5 and bottom 5 states for school infrastructure.

Your notes / observations:

Exercise 32 [Medium] — Ranking Districts Within Their Own State

Concepts practiced: `groupby().rank()`

125. Use `df.groupby('STATNAME')['literacy_column'].rank(ascending=False)` to rank every district within its own state by literacy.
126. Add this as a new column `STATE_LIT_RANK`.
127. Print all districts within one chosen state, sorted by `STATE_LIT_RANK`.
128. Explain why a district could be ranked #1 in a low-literacy state but still have lower absolute literacy than a mid-ranked district in a high-literacy state.

Your notes / observations:

Exercise 33 [Medium] — Stacked Bar Chart: Male vs Female Literacy

Concepts practiced: `plt.bar()` with `bottom=` • stacked bars

129. Select the 10 states with the highest average overall literacy.
130. Create a stacked (or grouped) bar chart comparing average male literacy and average female literacy for these 10 states.
131. Add a legend distinguishing 'Male Literacy' and 'Female Literacy'.
132. Rotate the x-axis state labels 45 degrees so they are readable.

Your notes / observations:

Exercise 34 [Medium] — Exploring District Names with String Operations

Concepts practiced: `str` accessor • `str[0]` • `value_counts()`

133. Use `df['DISTNAME'].str[0]` to extract the first letter of every district name.
134. Count how many district names start with each letter using `value_counts()`.
135. Create a bar chart of the 10 most common starting letters.
136. Use `df['DISTNAME'].str.len()` to find the longest and shortest district names in the dataset.

Your notes / observations:

Exercise 35 [Medium] — Fitting a Simple Trend Line

Concepts practiced: `np.polyfit()` • `np.poly1d()` • regression line on scatter

137. Create a scatter plot of total population (x) vs total number of schools (y).
138. Use `np.polyfit(x, y, 1)` to fit a straight line and `np.poly1d()` to build the line function.
139. Plot this fitted line on top of the scatter plot in a different color.

140. Print the slope and intercept and explain what the slope means in plain English (schools added per additional resident).

Your notes / observations:

PART 3 — HIGH COMPLEXITY (Exercises 36–50)

Goal: build composite indices, apply statistical reasoning (outliers, weighting, bootstrapping), and assemble full analytical dashboards — the kind of work a junior data analyst would be asked to do.

Exercise 36 [High] — Capstone Composite: Education Development Index

Concepts practiced: min-max normalization • vectorized numpy • weighted sum

141. Select 4-5 relevant numeric indicators: overall literacy, an infrastructure score (from Medium Exercise 31), inverse gender gap, and pupil-teacher ratio (inverted, since lower is better).
142. Min-max normalize each indicator to a 0-1 scale manually using NumPy: $(x - \min) / (\max - \min)$.
143. Combine the normalized indicators into a single Education Development Index using equal or chosen weights that sum to 1.
144. Rank all districts by this index and display the top 10 and bottom 10 districts with their component scores.

Your notes / observations:

Exercise 37 [High] — Multi-Column IQR Outlier Detection

Concepts practiced: np.percentile() • IQR formula • boxplots grid

145. For total population, total schools, and overall literacy, compute Q1, Q3 and IQR using np.percentile().
146. Flag a district as a multi-column outlier if it falls outside $[Q1 - 1.5 \cdot IQR, Q3 + 1.5 \cdot IQR]$ on at least 2 of the 3 columns simultaneously.
147. Print how many districts are flagged and list the 5 most extreme ones.
148. Create a 1x3 grid of boxplots for the three columns using plt.subplots(1,3), marking outliers visually.

Your notes / observations:

Exercise 38 [High] — Top-5 / Bottom-5 State Dashboard

Concepts practiced: plt.subplots(2,3) • multi-metric comparison

149. Rank all states by overall literacy and identify the top 5 and bottom 5.
150. Build a 2x3 grid of subplots comparing these 10 states across: literacy, sex ratio, dropout rate (if available), pupil-teacher ratio, total schools, and gender literacy gap.
151. Use consistent color coding (e.g. green for top 5, red for bottom 5) across all 6 panels.
152. Below the figure, print 3 bullet-style insight sentences describing the clearest pattern you observe.

Your notes / observations:

Exercise 39 [High] — Cross-Tabulating Literacy Band vs Infrastructure Band

Concepts practiced: pd.crosstab() • plt.imshow() heatmap

153. Reuse or recreate the LIT_BAND column (Low/Medium/High) from Medium Exercise 5.

154. Create a similar 3-band INFRA_BAND column from the infrastructure score built earlier.
155. Build a cross-tabulation of LIT_BAND against INFRA_BAND using `pd.crosstab()`.
156. Visualize the cross-tab as a heatmap using `plt.imshow()` with a colorbar, and annotate each cell with its count using `plt.text()` in a loop.

Your notes / observations:

Exercise 40 [High] — Simple vs Population-Weighted National Averages

Concepts practiced: `weighted average formula` • `np.sum()` • *Simpson's-paradox-style comparison*

157. Compute the simple (unweighted) national average literacy by averaging all district literacy values equally.
158. Compute the population-weighted national average literacy using `sum(literacy * population) / sum(population)`.
159. Repeat both calculations at the state level and store the results in a comparison DataFrame with columns SIMPLE_AVG and WEIGHTED_AVG.
160. Plot a bar chart showing, for the 10 states with the biggest gap between the two averages, both bars side by side, and explain in a comment which method you'd trust more and why.

Your notes / observations:

Exercise 41 [High] — Quadrant Analysis: Literacy vs Gender Gap

Concepts practiced: `df.apply(axis=1)` • *scatter with quadrant coloring*

161. Compute the national median literacy and the national median gender gap.
162. Write a function applied row-wise (`df.apply(..., axis=1)`) that assigns each district to one of 4 quadrants: High-Lit/Low-Gap, High-Lit/High-Gap, Low-Lit/Low-Gap, Low-Lit/High-Gap.
163. Count districts in each quadrant using `value_counts()`.
164. Create a scatter plot of literacy vs gender gap, colored by quadrant, with dashed lines at the median values marking the quadrant boundaries.

Your notes / observations:

Exercise 42 [High] — Population-Quantile Heatmap by State

Concepts practiced: `pd.qcut()` • *multi-level groupby* • *pivot + heatmap*

165. Use `pd.qcut()` to split districts into 4 population-size quantile bins (e.g. Small, Medium, Large, Very Large).
166. Group by STATNAME and population-quantile bin together, computing mean literacy for each combination.
167. Pivot this into a state-by-quantile-bin table.
168. Visualize the pivoted table as a heatmap with `plt.imshow()`, states on one axis and population-quantile bins on the other.

Your notes / observations:

Exercise 43 [High] — Annotated Correlation Heatmap

Concepts practiced: `df.corr()` • `plt.imshow()` • *nested loop annotation*

169. Select 8-10 key numeric variables across literacy, population, schools, teachers, and infrastructure.
170. Compute the full correlation matrix with `df.corr()`.
171. Display it using `plt.imshow()` with a diverging colormap (e.g. 'coolwarm') centered at 0, and a colorbar.
172. Use a nested for loop over rows and columns to annotate each cell with its correlation value using `plt.text()`, rounded to 2 decimals.

Your notes / observations:

Exercise 44 [High] — Bootstrap Confidence Interval for National Literacy

Concepts practiced: `np.random.choice()` • *resampling* • *percentile-based CI*

173. Write a loop that runs 1,000 iterations: each iteration draws a random sample (with replacement) of district literacy values using `np.random.choice()` and records the sample mean.
174. Store all 1,000 bootstrap means in a NumPy array.
175. Use `np.percentile()` to compute the 2.5th and 97.5th percentiles, giving a 95% confidence interval for national literacy.
176. Plot a histogram of the bootstrap means with vertical lines marking the confidence interval bounds.

Your notes / observations:

Exercise 45 [High] — Residual Analysis: Overperforming and Underperforming Districts

Concepts practiced: `groupby().transform()` • *residuals* • *diverging bar chart*

177. Compute each district's state-average literacy using `groupby('STATNAME')['literacy'].transform('mean')`.
178. Compute a residual column: district literacy minus its own state's average.
179. Sort by residual to find the 10 districts most above their state's average, and the 10 most below.
180. Create a horizontal diverging bar chart (positive residuals in one color, negative in another) for these 20 districts.

Your notes / observations:

Exercise 46 [High] — Automated Multi-State Insight Report

Concepts practiced: *for loops over groups* • *dictionary aggregation* • *f-string reporting*

181. Loop through every unique state, and for each one compute: number of districts, mean literacy, mean gender gap, and mean population, storing results in a dictionary of dictionaries.

182. Convert this dictionary into a summary DataFrame.
183. Print a formatted, human-readable report for the 5 states needing the most attention (lowest literacy combined with highest gender gap).
184. Create one horizontal bar chart showing all states sorted by mean literacy, with a reference line for the national average.

Your notes / observations:

Exercise 47 [High] — Manual Feature Scaling Comparison

Concepts practiced: min-max scaling • z-score standardization • overlaid histograms

185. Pick 5 numeric columns with very different scales (e.g. population in millions, literacy in percent, sex ratio in hundreds).
186. Manually implement Min-Max scaling (0 to 1) using NumPy - do not use sklearn.
187. Manually implement Z-score standardization (mean 0, std 1) using NumPy.
188. For one chosen column, plot 3 overlaid/side-by-side histograms: original, min-max scaled, and z-score scaled, to visually confirm the shape stays the same but the scale changes.

Your notes / observations:

Exercise 48 [High] — Data Quality Deep-Dive with Skewness Check

Concepts practiced: missing-data thresholds • manual skewness formula • conditional imputation

189. Identify all columns with more than 5% missing values and list them with their exact percentages.
190. For each numeric column with missing values, manually compute a simple skewness estimate using NumPy: $\text{mean}((x - \text{mean})^3) / \text{std}^3$.
191. Based on the skewness value, decide and justify (median for skewed columns, mean for roughly symmetric columns) an imputation strategy for each column.
192. Apply your chosen imputation and confirm with `df.isnull().sum()` that no missing values remain in those columns.

Your notes / observations:

Exercise 49 [High] — Investigating a Simpson's Paradox Candidate

Concepts practiced: subgroup comparison • `pd.qcut()` • conditional logic across groups

193. Choose 2 states where one has a higher overall (state-wide) average literacy than the other.
194. Split all districts into population-quantile subgroups (as in High Exercise 7) within each of the 2 states.
195. Compare the two states' average literacy within each matching population-quantile subgroup.
196. Determine and report whether the state with the lower overall average actually has higher literacy in every subgroup (a genuine Simpson's paradox) or not, using printed evidence.

Your notes / observations:

Exercise 50 [High] — Capstone Dashboard: One-Page Education Insights Infographic

Concepts practiced: `plt.figure` + `GridSpec` • 6+ combined chart types • narrative summary

197. Using matplotlib's `GridSpec` (or `plt.subplots` with a custom grid), build a single figure containing at least 6 different charts: a bar chart of top/bottom states, a histogram of literacy, a scatter of population vs schools, a boxplot of literacy by region, a pie chart of district share by state, and a correlation heatmap.
198. Give the overall figure a subtitle such as 'India District-Wise Education: 2015-16 Insights'.
199. Ensure consistent color themes and readable font sizes across all 6 panels using `plt.rcParams` or per-axis styling.
200. Below the figure, write a 5-6 sentence narrative summary of the most important findings from this entire workbook, written for a non-technical policymaker audience.

Your notes / observations:

Wrap-Up & Next Steps

By completing all 50 exercises you will have practiced the full core toolkit needed for real-world data analysis: loading and cleaning data, exploring it with pandas, computing with NumPy, and communicating findings with matplotlib.

Suggested next steps: try the same exercises on a different districtwise year if available, recreate the Part 3 dashboard using seaborn or plotly for extra polish, or extend the Education Development Index (Exercise 36) with your own weighting scheme and defend your choices.